

UE5 Listen Server 기반 멀티플레이 프로젝트

EternalReturn — Unreal Engine 모작



이터널 리턴(Eternal Return)을 Unreal Engine 5.7로 재현한 포트폴리오 프로젝트입니다.

목차

- 개요
- 기술 스택
- 아키텍처
- 구현 내용
- Core System
 - 스킬 컴포넌트
 - 인벤토리 컴포넌트
 - 구조물 시스템
 - 시야 / 포그오브워 시스템
 - 캐릭터 선택 → 인게임 전환
 - TCP 회원가입 / 로그인
 - 방 시스템
 - Crafting 컴포넌트
- 트러블슈팅 / 배운 점
 - ① Seamless Travel
 - ② OnRep 호출 순서

- [스크린샷 / 영상](#)

개요

항목	내용
장르	탑다운 서바이벌 액션 (Battle Royale + MOBA 하이브리드)
엔진	Unreal Engine 5.7
언어	C++ / Blueprint 혼합
네트워크	Listen Server + 자체 TCP 백엔드
개발 기간	2026.05 ~ 2026.07
개발 인원	1인

프로젝트 소개

- 평소 즐겨 하던 게임 이터널 리턴을 플레이하면서 **"이 게임은 어떤 구조로 만들어졌을까?"** 라는 궁금증에서 시작한 프로젝트
- 전투/스킬, 인벤토리, 시야 시스템, 몬스터 AI, 로비/매칭 등 시스템을 직접 설계·구현
- 멀티플레이어 게임 개발 파이프라인 전반을 경험하는 것을 목표로 개발
- 특히 이터널 리턴 특유의 **시야 기반 정보전**과 **전투 시스템** 구현에 집중

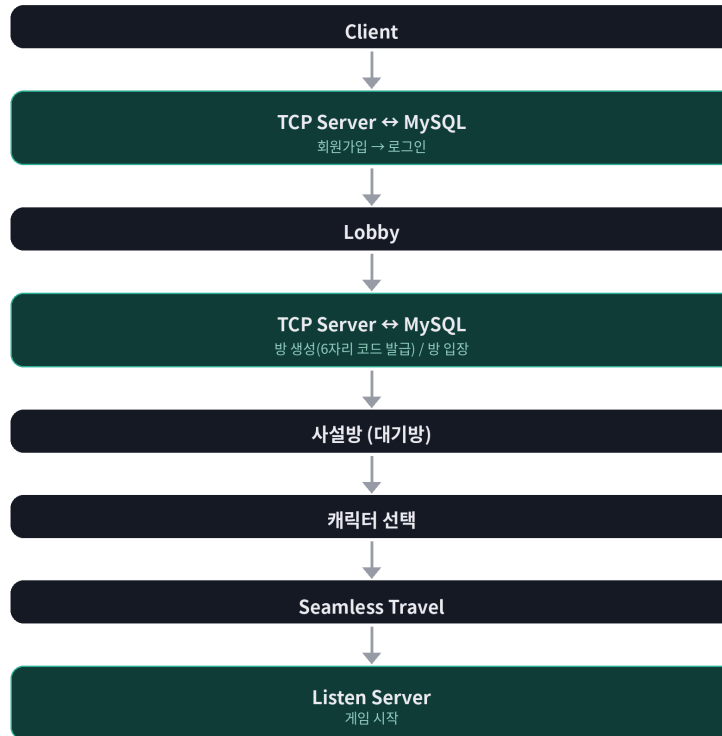
목표 이를 통해 네트워크, 게임플레이, UI, 데이터 관리까지 멀티플레이어 게임 개발 전 과정을 경험하고, 실제 상용 게임에서 사용하는 구조를 직접 설계·구현하는 것을 목표로 했습니다.

기술 스택

- **C++** — 핵심 시스템 (스탯/컴포넌트 계층, 스킬 로직, 네트워크 리플리케이션)
- **Blueprint** — 캐릭터별 스킬 연출, UI, 레벨 로직
- **TCP 백엔드** — 별도 C++ 콘솔 앱, Winsock2 + nlohmann/json + MySQL 8.4
- **Replication System** — Listen Server 기반 상태 동기화 (OnRep 중심)
- **Behavior Tree / NavMesh** — 몬스터 AI, 경로 탐색 및 클릭 이동

아키텍처

로그인/로비/룸 매칭은 UE 네트워크 세션이 아닌 별도 TCP 백엔드로 처리 실제 게임 플레이는 Listen Server로 동작



- 메인 로비는 TCP 클라이언트 화면으로만 동작 (UE 네트워크 세션 아님)
- 호스트가 캐릭터 선택 레벨에 진입하는 시점에 Listen Server 시작
- 클라이언트는 6자리 방 코드 입력 → 백엔드가 호스트 IP/Port 반환 → **OpenLevel** 접속
- 범위: LAN 플레이 (NAT 트래버설은 추후 과제)

구현 내용

캐릭터 / 전투 시스템

- **UBaseStatComponent** 계층으로 캐릭터/몬스터 스탯 공통화 HP/공격력/방어력 리플리케이션
- 방어력 공식 등 원작 전투 수치 재현

스킬 시스템

- **USkillComponent** 가 쿨타임·스킬 레벨·사용 가능 여부 전담 실제 연출은 캐릭터별 Blueprint에서 오버라이드
- **UDA_SkillBase** DataAsset
의 **SkillType** Enum(Projectile/Dash/AreaDamage/Buf/CC/Toggle/Passive)으로 스킬 종류 분기
- Q/W/E/R(5레벨) · T(패시브 3레벨) · D(무기 스킬) · F(전술 스킬) 원작 키 구조 재현

인벤토리 / 장비 시스템

- 10슬롯 인벤토리 + 5슬롯 장비, **OnRep** 기반 리플리케이션으로 UI 동기화
- 무기 타입 제한, 장착 시 스탯 연동

구조물 / 루팅 시스템

- `ItemList(Replicated)` 는 구조물이 소유, 추가/제거는 `GameMode`가 경유 처리
- 루팅 흐름: 슬롯 클릭 → `ServerTakeItem` → `TakeItemFromStructure` → `InventoryComponent.AddItem`

시야 / 포그오브워

- `SceneCaptureComponent2D` + 렌더타겟으로 시야 밖 월드 암전
- `VisionDetectionComponent` 의 스피어 오버랩으로 시야 범위 밖 액터 은닉

몬스터 AI

- Behavior Tree (BB/BT/BTS_Monster) 기반 추격, HP 기반 귀환 로직

캐릭터 선택 → 인게임 전환

- Seamless Travel 기반 캐릭터 선택 → 스폰 파이프라인

제작 / 루팅 시스템

- `UCraftingComponent` 기반 제작, 구조물 루팅 UI

로그인 / 로비

- TCP 백엔드 연동 로그인/회원가입 UI `BackendConnectionSubsystem` 으로 레벨 전환 간 TCP 연결 유지

TCP 채팅

| 내용 추후 추가 예정

Core System

스킬 컴포넌트

설계 의도 캐릭터마다 다른 스킬 로직을 갖더라도 하나의 공용 컴포넌트로 관리

구조

스킬 입력 → 서버에 스킬 사용 요청 → 데이터에 지정된 스킬 종류에 따라 분기 → 해당 스킬 실행

구현 포인트

- DataAsset: Q/W/E/R 스킬마다 스킬 이름, 종류, 재사용 대기시간, 소모 마나, 사거리, 피해량/회복량/보호막 수치, 발사체 종류, 군중제어(CC) 부여 여부 등 스킬에 필요한 모든 수치와 설정을 데이터로 저장
- TargetLocation: 마우스 클릭 위치를 `ExecuteSkill` → `UseWeaponSkill/UseTacticalSkill` → `SkillType()` 까지 7개 타입별 함수에 하나의 파라미터로 관통
- SkillType 분기: Projectile / Dash / AreaDamage / Buff / CC / Toggle / Passive 7가지로 분류



인벤토리 컴포넌트

설계 의도 서버 권위를 유지하면서 UI가 자동으로 동기화되도록 함

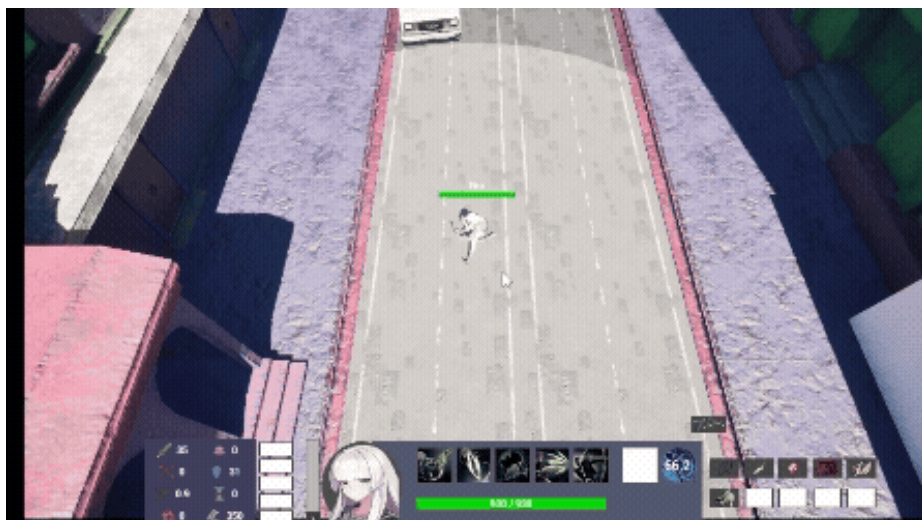
동작 흐름

서버에서 인벤토리 변경 → 클라이언트에 변경 사항 동기화 → UI에 자동 반영

구현 포인트

- InventorySlots/EquipSlots는 Replicated 변수
- OnRep_InventorySlots/OnRep_EquipSlots가 값 변경 시 자동 호출
- OnInventoryUpdated/OnEquipSlotsUpdated 델리게이트를 WBP_HUD의 Event Construct에서 바인딩
- Host에서는 OnRep이 자동 호출되지 않아 별도 처리가 필요

? 왜 Multicast 대신 OnRep을 사용했는가? Multicast는 서버가 모든 클라이언트에 무조건 뿌리는 구조라 결합도가 높아지는 반면, OnRep은 값이 실제로 바뀐 클라이언트에서만 자동 호출되고 서버 로직과 분리할 수 있어 더 적합하다고 판단



구조물 시스템

설계 의도 루팅 가능한 구조물(상자 등)의 아이템 목록을 서버 권위로 관리하고, 여러 클라이언트가 동시에 열어도 UI가 동기화되도록 함

구조

슬롯 클릭 → 서버에 아이템 획득 요청 → GameMode가 목록에서 제거 + 인벤토리에 추가 → UI 자동 갱신

구현 포인트

ItemList(Replicated)는 구조물이 소유, OnRep으로 UI 자동 갱신 추가/제거는 GameMode가 처리 (구조물은 Owner 불명확 → RPC 라우팅 문제 방지) 흐름: 슬롯 클릭 → ServerTakeItem → TakeItemFromStructure(GameMode) → ItemList 제거 + InventoryComponent.AddItem WBP_LootBox 는 StructureRef 기준으로 바인딩/언바인딩



시야 / 포그오브워 시스템

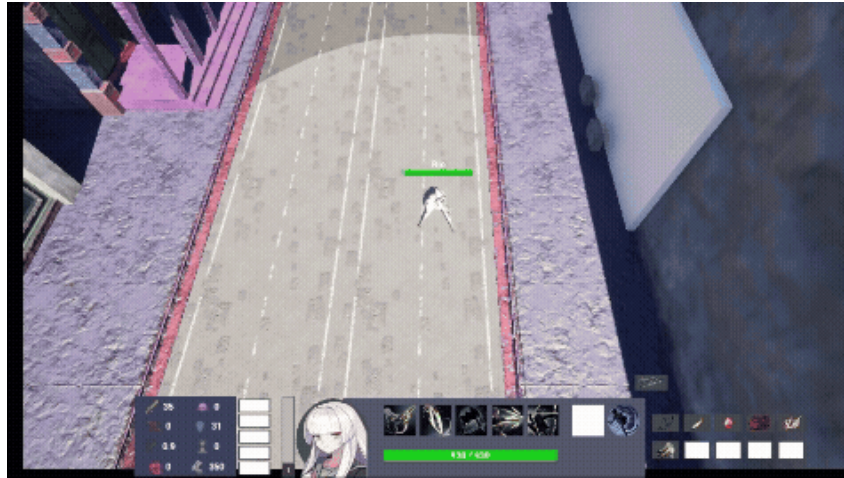
설계 의도 이터널 리턴 특유의 시야 기반 정보전을 재현 — 시야 밖 정보를 차단

구조

플레이어 시야 범위 계산 → 범위 안에 있는지 확인 → 있으면 보이게, 없으면 숨김

구현 포인트

- SceneCaptureComponent2D + RenderTarget: 시야 범위를 캡처해 월드 암전 처리
- SphereOverlap: 시야 구체와 겹친 액터를 보이게/숨기게 처리
- IsLocallyControlled: 각 클라이언트가 자기 화면 기준으로만 판정하도록 게이팅



캐릭터 선택 → 인게임 전환

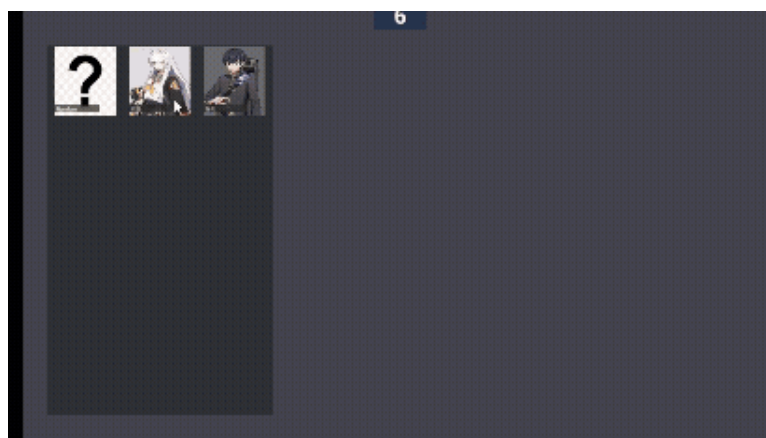
설계 의도 캐릭터 선택 상태를 다른 플레이어와 공유하고, 인게임 진입 시 정확히 반영되도록 함

캐릭터 선택 흐름

캐릭터 클릭으로 선택 요청 → 서버에 선택한 캐릭터 정보 전달 → PlayerState에 선택 정보 저장 → 다음 레벨로 이동 준비 → 이동 후 선택 정보를 유지한 채 캐릭터 스폰

구현 포인트

- Seamless Travel: 레벨 전환 시 로딩 화면 없이 다음 레벨로 자연스럽게 이동시키는 트래블 방식
- 레벨 전환 중에도 월드가 항상 하나는 떠 있어야 한다는 엔진 제약으로 Transition Map 사용
- HandleStartingNewPlayer: 신규 접속과 Seamless Travel 모두에서 호출되는 스폰 훅 (PostLogin은 호출 안 됨)
- CopyProperties: 캐릭터 선택 정보를 다음 레벨에서도 유지하도록 처리



TCP 회원가입 / 로그인

설계 의도 인증(계정) 처리와 게임 세션(전투·리플리케이션) 처리를 완전히 분리하기 위해 별도 TCP 백엔드 구축 구조

클라이언트 로그인 요청 → 서버에서 DB 조회 → 로그인 성공/실패 응답 → 클라이언트에 결과 반영

구현 포인트

- DatabaseManager: mutex로 동시 요청 보호, mysql_real_escape_string으로 SQL Injection 방지
- BackendConnectionSubsystem(GameInstanceSubsystem)이 델리게이트로 비동기 응답 수신, 레벨 이동 후에도 유지

Crafting 컴포넌트

설계 의도 인벤토리가 바뀔 때마다 제작 가능 목록은 자동으로 계산되게 하고, 실제 제작 진행은 서버가 시간을 재며 권한을 갖고 처리하도록 분리

구조 1 — 제작 가능 목록 갱신

인벤토리 변경 감지 → 서버가 보유 중인 재료 확인 → 조합 가능한 아이템 목록 갱신 → 클라이언트에 동기화

구조 2 — 제작 진행

제작 시작 요청 → 서버가 아이템 등급에 따라 제작 시간 결정 → 클라이언트에 게이지 UI 표시 요청 → 시간 경과 후 재료 소모 + 결과물 지급

구현 포인트

- 재료 두 가지를 모두 갖고 있는 조합만 골라서 제작 가능 목록을 자동으로 추림
- 아이템 등급이 높을수록 제작 시간이 길어지도록 6단계로 차등 설정
- 제작 중인지 여부를 서버가 들고 있어서, 이동 등으로 취소될 때도 클라이언트가 임의 판단하지 않고 서버 상태를 그대로 따름
- 목록 갱신은 서버 자신에게는 자동 반영되지 않아서, 서버에서 직접 갱신 알림을 호출
- 컴포넌트가 아니라 PlayerController를 거쳐서 게이지 UI 시작을 클라이언트에 알림

트러블슈팅 / 배운 점

① Seamless Travel

항목	내용
문제	캐릭터 선택 → 인게임 전환 시 Possess(스폰) 자체가 안 되고 SelectName도 초기화됨
원인	스폰(Possess) 로직이 PostLogin에 있었는데, Seamless Travel로 넘어온 플레이어에게는 호출되지 않음
해결	스폰(Possess) 로직을 HandleStartingNewPlayer로 이전 (신규 접속·Seamless Travel 공통 대응)
결과	신규 접속, Seamless Travel 양쪽 모두 캐릭터 선택 정보가 정확히 유지된 채 스폰됨

💡 **배운 점** 엔진 라이프사이클 혹마다 호출 시점과 대상이 다르다는 것 — PostLogin, HandleStartingNewPlayer, OnRestartPlayer 각각의 호출 조건을 정확히 알아야 함

② OnRep 호출 순서

항목	내용
문제	HP는 정상적으로 Replication 되지만 UI가 갱신되지 않는 문제 발생
원인	OnRep 함수가 호출되는 시점보다 UI의 Delegate 바인딩이 늦게 수행되어 Broadcast를 수신하지 못함. Listen Server에서는 호스트 자신에게는 OnRep이 자동 호출되지 않는 특성도 함께 확인
해결	Delegate 바인딩 시점을 조정하여 OnRep 호출 이전에 구독이 완료되도록 수정. 호스트의 경우에는 서버에서 값을 변경한 직후 직접 Broadcast를 호출하도록 처리
결과	Host와 Client 모두 동일한 시점에 UI가 정상 갱신되도록 개선

💡 **배운 점** Replication만 이해하는 것이 아니라 OnRep 호출 시점과 객체 초기화 순서까지 함께 고려해야 한다는 것을 배웠습니다.